

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

B.Tech. | BCSE497J – Project-I

AI-Driven Post-Quantum Cryptography Migration Framework

Team members:

ARPIT MITTAL (23BCI0044)

PRIYANSHU SEN (23BCT0040)

VEDANSH TRIVEDI (23BCT0056)

Faculty guide:

BADRINATH N, Associate Professor Grade 2

Team ID: 20106UG02

Approval Mail From Guide

- *Get the approval for the following points from your guide through mail before 21-10-2026.(sample attached below)*

The Fall Semester Project Review 3 for Project ID <<25000UG01>> has been approved.

- **Meeting with Guide Frequently:** Satisfied / Not Satisfied
- **PowerPoint Presentation:** Approved / Not Approved
- **Project Demonstration:** Approved / Not Approved
- **Final Project Report:** Satisfied / Not Satisfied
- **Project Outcome:** Ongoing / Completed
- **Turinitin PlagiarismPercentage : ____** %
- **Turinitin AI Score : _____** (should be *% or less than 25%)

All the above aspects are addressed and updated by the project team.

Aim

- To automate the migration of classical public-key cryptography in existing Python software to NIST-standardised post-quantum algorithms, and to prove the migrated code still works.
  - Detect every use of RSA, ECC and Diffie-Hellman in a codebase
  - Recommend the correct NIST replacement for each occurrence
  - Rewrite the source automatically onto ML-KEM and ML-DSA
  - Verify the result by executing the migrated application
  - Report the measured cost of migration in size and time

Abstract

- Shor's algorithm breaks RSA, ECC and Diffie-Hellman; NIST standardised the replacements in August 2024 (FIPS 203, 204, 205).
- Standardising algorithms solved only half the problem — migrating existing codebases is still slow, manual and needs cryptographic expertise.
- We built an end-to-end framework: AST detection → purpose inference → NIST recommendation → rule-based rewriting → functional verification → reporting.
- Evaluated on a 29-module labelled benchmark with 38 annotated sites: precision 1.00 and recall 1.00 over 31 in-scope sites, purpose accuracy 1.00, zero false alarms on 6 clean modules.
- 9 of 9 sites migrated across 3 sample applications; the migrated RSA and ECDSA applications execute successfully under real ML-KEM and ML-DSA.
- Measured cost: ML-KEM-768 keys are 4.03× RSA-2048; ML-DSA-65 signatures are 46.6× ECDSA P-256.

Literature Review

Paper / Source	Objective	Methodology	Pros / Cons	Findings
NIST FIPS 203, 204, 205 (2024)	Standardise quantum-resistant KEM and signature schemes	Multi-round public evaluation of lattice and hash-based candidates	Pro: settles which algorithms to adopt. Con: silent on migrating existing code	ML-KEM, ML-DSA and SLH-DSA with defined parameter sets per security category
NCCoE, NIST SP 1800-38 — Migration to PQC	Define a migration programme for enterprises	Practice guidance centred on cryptographic discovery and inventory	Pro: establishes discovery as the first step. Con: produces an inventory only	Knowing where crypto lives does not say what should replace it
Stebila & Mosca, Open Quantum Safe (SAC 2016); kyber-py, dilithium-py	Make post-quantum primitives usable in real software	Open-source reference and optimised implementations	Pro: production-quality primitives available. Con: supplies the destination, not the journey	Primitives are available and interoperable well ahead of adoption
Chen et al., Evaluating LLMs trained on code (2021)	Assess model-driven source-to-source transformation	Benchmark generated code against functional tests	Pro: handles code shapes rules cannot match. Con: output requires human verification	A plausible patch is indistinguishable from a correct one without execution
Mosca (2018); Barker et al., NIST white paper (2021)	Frame the urgency and organisational readiness problem	Risk analysis of the harvest-now-decrypt-later threat	Pro: justifies acting before quantum hardware exists. Con: no technical migration path	Long-lived data is already exposed; migration cannot wait for the threat

Research Gap

- Discovery tools identify cryptography but do not recommend replacements.
- Standards specify replacements but do not transform code.
- Implementations provide primitives but do not migrate anything.
- Transformation research changes code but does not verify the result.
- **No existing system connects detection → decision → transformation → verification into one workflow.**
- **Verification is the widest part of the gap: cryptographic correctness is not visible by inspection — it must be demonstrated by execution.**

Objectives

- Detect RSA, ECC and DH usage with file, line, algorithm and library.
- Classify each occurrence as key encapsulation, key exchange or signature.
- Recommend a specific NIST parameter set with a written justification.
- Rewrite source onto PQC without modifying the originals.
- Verify by round-trip tests, negative tests and executing migrated code.
- Generate JSON, Markdown and HTML reports with a measured benchmark.
- Evaluate detection quantitatively against a labelled benchmark.

Identified Sustainable Development Goals: SDG 9 – Industry, Innovation and Infrastructure (primary); SDG 16 – Peace, Justice and Strong Institutions (secondary)

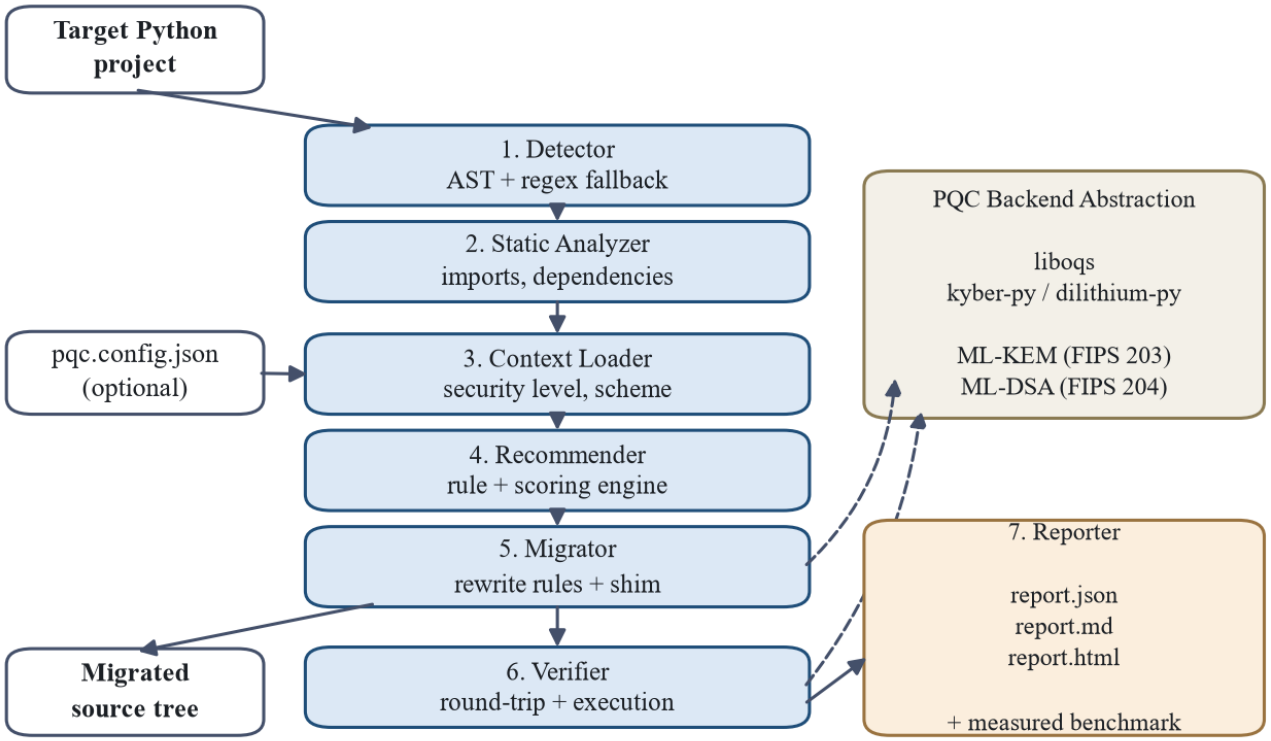
Outcome: Conference (Scopus) — paper drafted, target venue to be finalised with the guide





Framework/Architecture/Block Diagram

- Seven independent stages; each communicates only through explicit data records, so no stage can report an outcome an earlier stage did not produce.



Functional Requirements

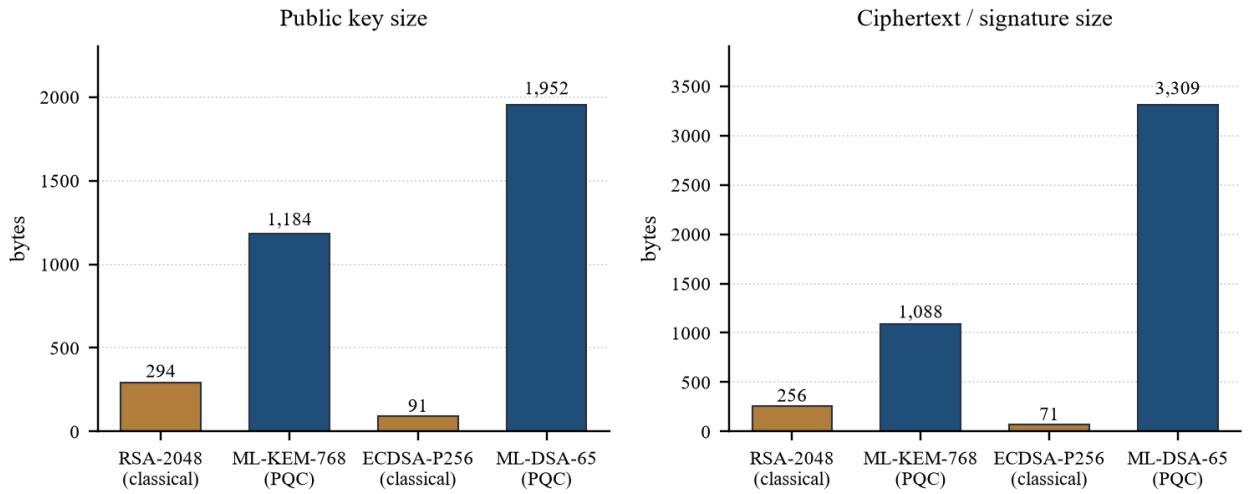
- FR1 — Recursively scan a project and report every RSA, ECC and DH usage.
- FR2 — Report file, line, algorithm family, library and matched construct.
- FR3 — Classify each occurrence as KEM, key exchange or signature.
- FR4 — Continue analysing files that fail to parse, via a textual fallback.
- FR5 — Accept target security category and signature family from config or CLI.
- FR6 — Recommend a NIST parameter set per finding, with a justification.
- FR7 — Produce migrated copies without modifying the original source.
- FR8 — Never report a finding as migrated unless a rule actually applied.
- FR9 — Verify by round-trip and negative tests, and execute migrated apps.
- FR10 — Generate JSON, Markdown and HTML reports with a live benchmark.

Modules

- Detector — AST walk over every module, matched against a pattern index; alias resolution; regex fallback for unparseable files.
- Purpose classifier — re-infers the role of key generation from how the key is used elsewhere in the same module.
- Recommender — purpose selects the family, security category selects the parameter set, scoring raises it from classical strength signals.
- Migrator — independent rewrite rules plus a compatibility shim that keeps the classical object model over ML-KEM / ML-DSA.
- PQC backend — one interface over liboqs and pure-Python kyber-py / dilithium-py; unsupported algorithms raise rather than substitute.
- Verifier — round-trips, negative tests, and execution of migrated apps.
- Reporter — JSON, Markdown, HTML with a benchmark measured at run time.

Experiments and Results

- **Detection benchmark — 29 modules (18 in-scope, 5 adversarial, 6 clean), 38 annotated sites.**
- In-scope: precision 1.000, recall 1.000, F1 1.000 over 31 sites; purpose accuracy 1.000; 0 false alarms on 6 clean modules.
- Extended (adversarial): recall 0.000 of 7 — wrapper libraries, string parameters and dynamic dispatch bound syntactic analysis.
- Pipeline: 9/9 sites migrated across 3 apps; migrated RSA and ECDSA apps execute successfully. 70 automated tests pass.



Conclusion and Future Work

- **PQC migration is a software-engineering problem with a cryptographic correctness criterion — not just an algorithm choice.**
- Delivered an end-to-end verified pipeline: detection, purpose inference, recommendation, rewriting, verification and reporting.
- All seven objectives met; limitations measured and reported rather than concealed.
- **Future work**
  - LLM-assisted transformation for the 7 adversarial sites rules cannot reach — verification stage stays, so generated code must still execute.
  - Protocol-level restructuring of Diffie-Hellman handshakes.
  - Wrapper-library coverage, languages beyond Python, the optimisation module, and independent evaluation on real repositories.

Outcome Achieved

- **Outcome identified: International Conference Proceedings (Scopus)**
- Status: full paper drafted — 8 sections, 5 result tables, measured evaluation. Target venue to be finalised with the guide.
- Title: “From Detection to Verified Replacement: An End-to-End Framework for Post-Quantum Cryptographic Migration of Python Codebases”
- *[Replace this slide with the submission / acceptance screenshot once the paper is submitted, and upload the proof to the given link before 27 Oct 2026.]*

References

[1] National Institute of Standards and Technology, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, Gaithersburg, MD, USA, Aug. 2024.

[2] National Institute of Standards and Technology, *Module-Lattice-Based Digital Signature Standard*, FIPS 204, Gaithersburg, MD, USA, Aug. 2024.

[3] National Institute of Standards and Technology, *Stateless Hash-Based Digital Signature Standard*, FIPS 205, Gaithersburg, MD, USA, Aug. 2024.

[4] National Cybersecurity Center of Excellence, *Migration to Post-Quantum Cryptography*, NIST Special Publication 1800-38, Gaithersburg, MD, USA.

[5] G. Pope, "kyber-py: A pure-Python implementation of ML-KEM (FIPS 203)," software repository, version 1.2.0.

[6] G. Pope, "dilithium-py: A pure-Python implementation of ML-DSA (FIPS 204)," software repository, version 1.4.0.

[7] W. Barker, W. Polk and M. Souppaya, *Getting Ready for Post-Quantum Cryptography*, NIST Cybersecurity White Paper, 2021.

[8] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint* arXiv:2107.03374, 2021.

[9] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38-41, 2018.

[10] D. Stebila and M. Mosca, "Post-quantum key exchange for the Internet and the Open Quantum Safe project," in *Proc. Selected Areas in Cryptography (SAC 2016)*, LNCS vol. 10532, pp. 14-37.

[11] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484-1509, 1997.